

Lab Report: Week 3 Session B

Test-Driven Development Practice

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 3 Session B on **test-driven development (TDD)** with OpenCode Desktop: Red (failing tests before implementation), Green (minimum code to pass), and Refactor (improve structure while tests stay green). The SCS-CN function `calculate_runoff(P, CN)` was developed in `week3_session.b/` separately from the Session A scaffold. Red phase produced `ModuleNotFoundError`; Green and Refactor each yielded **7 passed** tests. Sanity checks $Q(50, 80) \approx 13.80$ mm and $Q(80, 85) \approx 43.55$ mm match earlier course labs. Evidence: six screenshots (OpenCode and terminal per phase) and full source in Appendix ??-??.

1 Introduction

TDD treats tests as a **contract** for correct behavior: write tests first (Red), implement until they pass (Green), then refactor without changing outputs. This reduces wrong formulas and plausible-but-false AI outputs. Session B applies TDD to the same SCS-CN physics as Session A, but in a fresh folder `week3_session.b/` with tests written *before* `src/runoff.py` existed.

2 Objectives

- Apply the Red $\rightarrow$ Green $\rightarrow$ Refactor cycle with AI assistance.
- Write comprehensive pytest tests before implementation.
- Use tests to validate AI-generated hydrology code.
- Document the process in `prompt_log.md`.

3 Methods

3.1 Conventions (all phases)

$S = 25400/\text{CN} - 254$ mm, $I_a = 0.2S$, $Q = 0$ if $P \leq I_a$, else $Q = (P - I_a)^2 / (P - I_a + S)$, with $0 \leq Q \leq P$. Invalid $P < 0$ or $\text{CN} \notin [1, 100]$ raise `ValueError`. Python 3.8 compatible hints (`Union`, not `X | Y`).

3.2 Exercise 1: Red phase

Prompt `prompt_red.txt` asked OpenCode for `tests/test_runoff.py` only—seven cases: normal ($P = 50$, $\text{CN} = 80$), $P = 0$, $P < I_a$, $\text{CN} = 100$, negative P , invalid CN , and a 7×7 grid asserting $Q \leq P$. No `src/runoff.py` was created.

3.3 Exercise 2: Green phase

Prompt `prompt_green.txt` included the full test file; OpenCode implemented `src/runoff.py` to satisfy all tests.

3.4 Exercise 3: Refactor phase

Prompt `prompt_refactor.txt` requested readability and clearer errors without changing numerical results; `pytest` was run after refactoring.

4 Results

4.1 Submitted file inventory

Table 1: Files in `week3_session_b/`.

Path	Role
<code>tests/test_runoff.py</code>	Seven pytest cases (written in Red)
<code>src/runoff.py</code>	SCS-CN implementation (Green, refactored)
<code>prompt_red.txt</code> , <code>prompt_green.txt</code> , <code>prompt_refactor.txt</code>	AI prompts
<code>prompt_log.md</code>	TDD documentation
<code>requirements.txt</code>	pytest dependency

4.2 Red phase

Terminal and OpenCode confirm collection failure: `ModuleNotFoundError: No module named 'src.runoff'` (Figures ??-??).

4.3 Green phase

All seven tests passed on first implementation (Figures ??-??). Hand-checked: $Q(50, 80) \approx 13.80$ mm; $Q(80, 85) \approx 43.55$ mm.

4.4 Refactor phase

Refactoring introduced named constants, validation helpers, and `_compute_runoff`; public `calculate_runoff` became a short `validate`→`compute`→`clamp` pipeline. Tests remained green (Figures ??-??).

Table 2: Refactoring changes (behavior unchanged).

Change	Rationale
<code>_RETENTION_FACTOR</code> , etc.	Remove magic numbers
<code>_validate_positive_rainfall</code> , <code>_validate_curve_number</code>	Separate validation; clearer errors
<code>_compute_runoff</code>	Pure math on validated floats
<code>effective_rainfall</code>	Clearer than repeated $P - I_a$
Short <code>calculate_runoff</code>	Readable public API

Table ?? summarizes the TDD cycle.

5 Analysis and validation

Tests as contract: The normal-case expected value (≈ 13.80 mm) and impervious case ($Q = P$ when $CN = 100$) encode physics; a wrong S formula or missing $P \leq I_a$ branch would fail

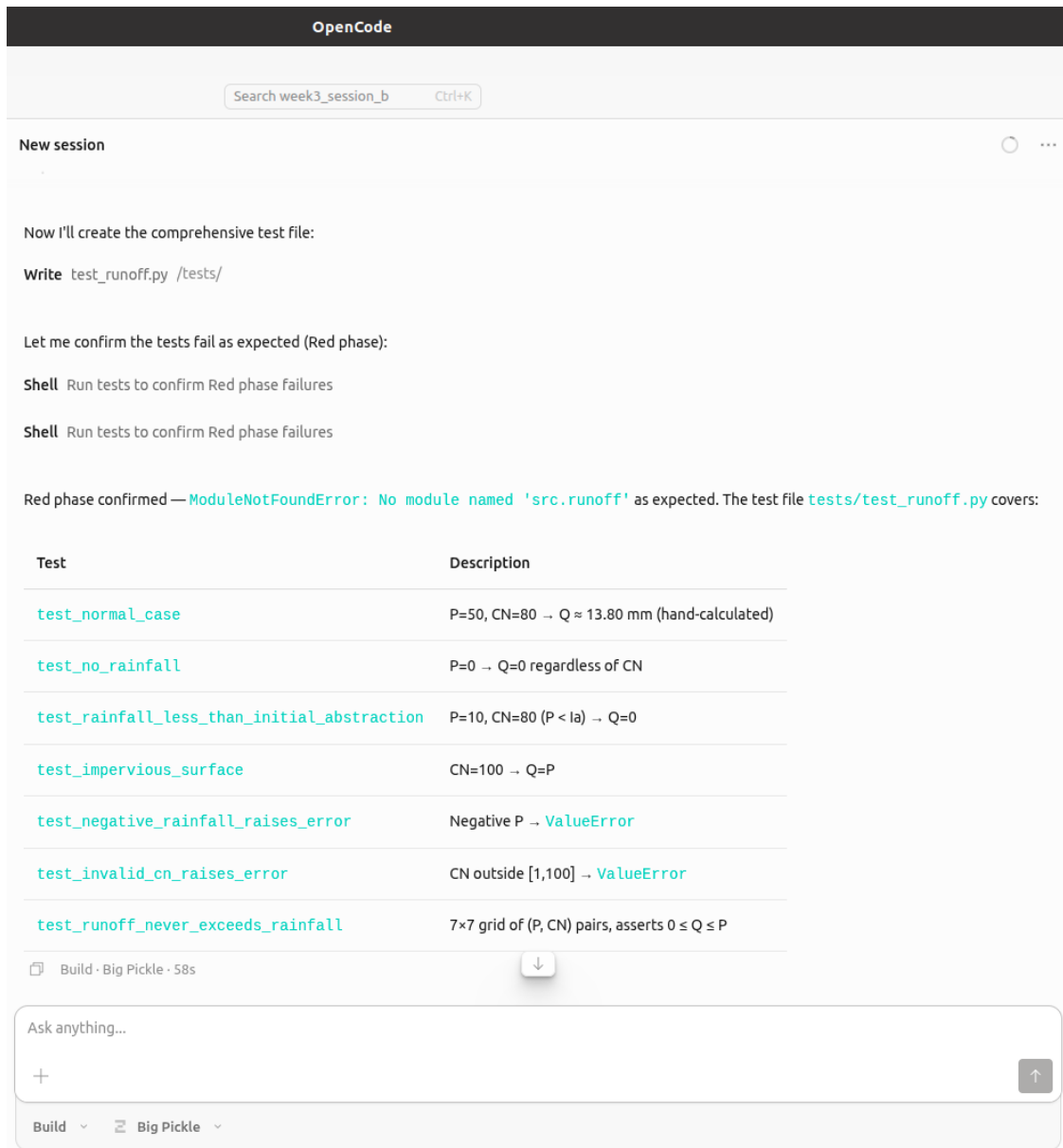


Figure 1: OpenCode Red: generated test_runoff.py; confirmed import error.

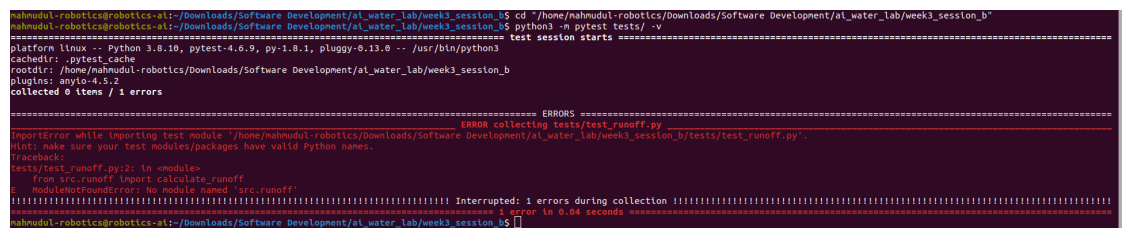


Figure 2: Terminal Red: pytest -v — ModuleNotFoundError (expected).

Table 3: TDD cycle outcomes.

Phase	src/runoff.py	pytest
Red	Absent	Import error
Green	Present (monolithic)	7 passed
Refactor	Helpers + constants	7 passed

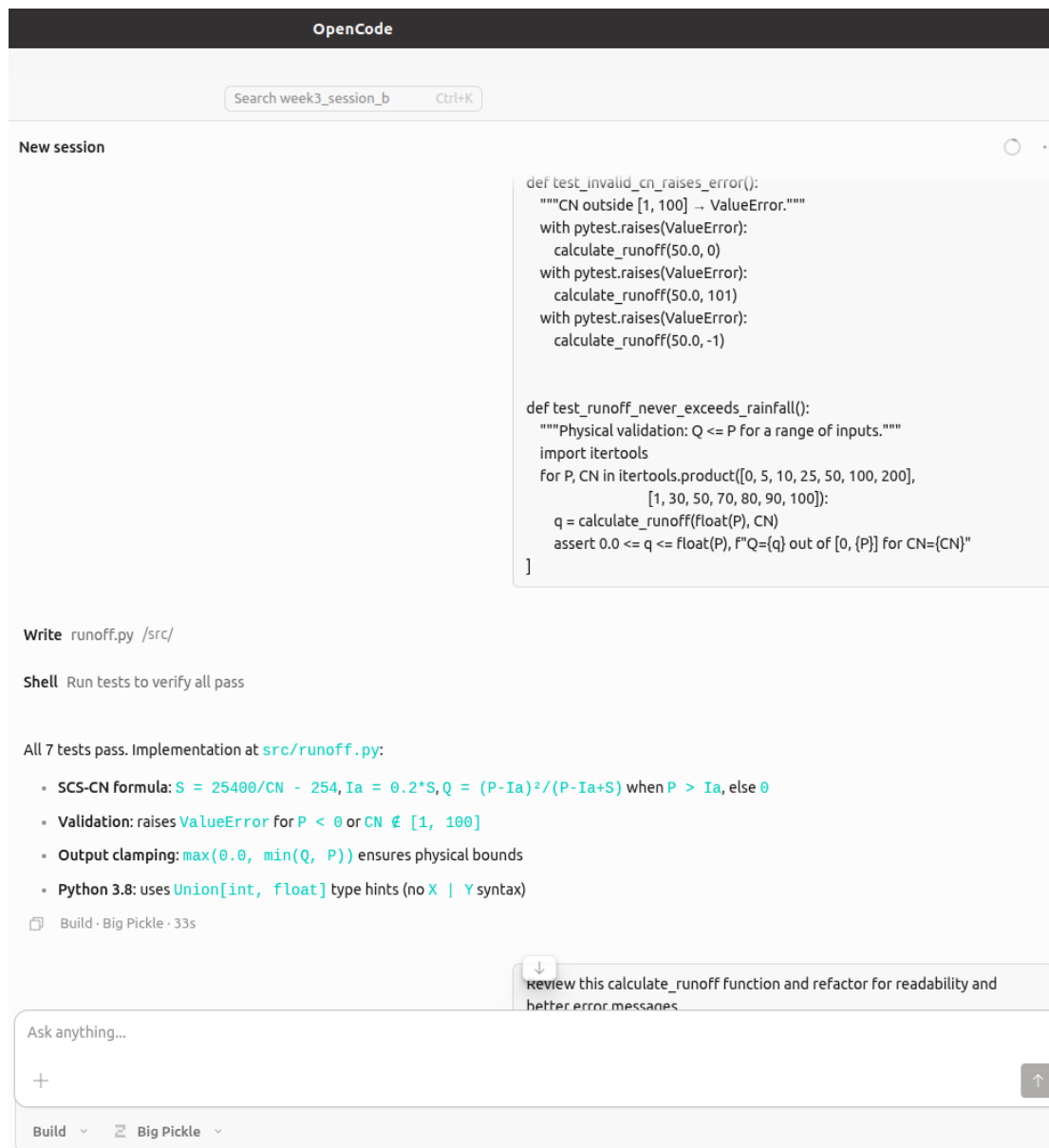


Figure 3: OpenCode Green: `src/runoff.py` created; 7 tests pass.

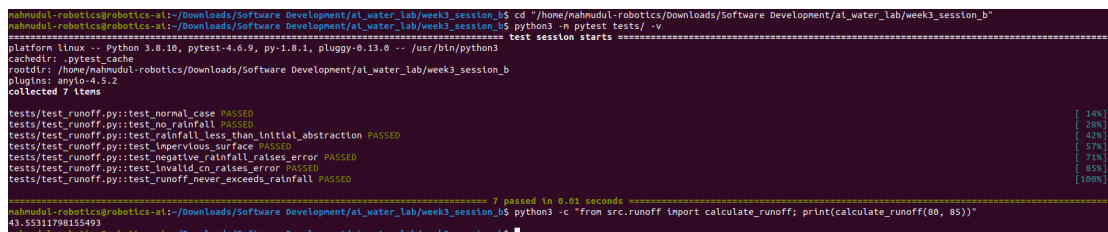


Figure 4: Terminal Green: 7 passed.

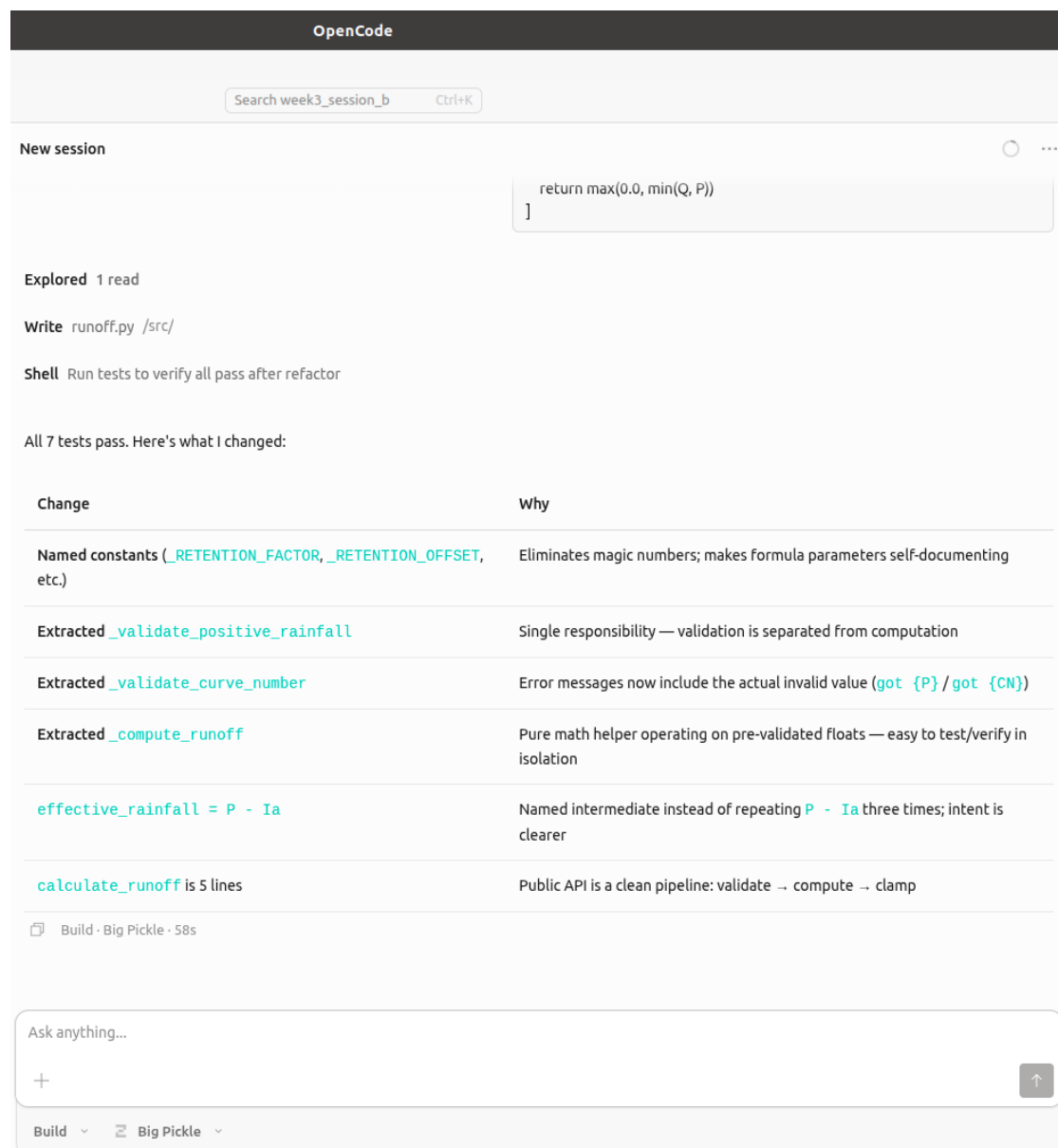


Figure 5: OpenCode Refactor: structural improvements; tests still pass.

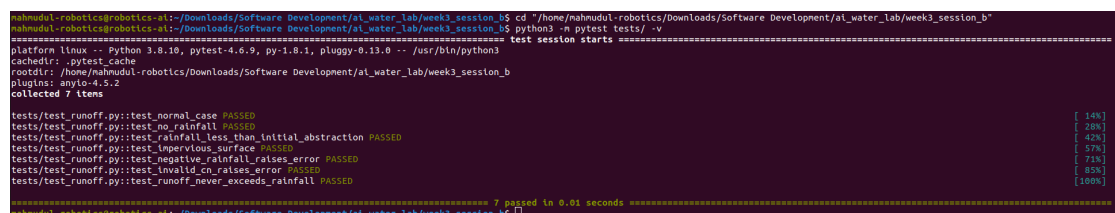


Figure 6: Terminal Refactor: 7 passed after refactor.

immediately.

AI role: OpenCode generated correct tests and implementation; refactor improved maintainability without changing outputs. Local `pytest` was still run to verify agent claims.

Session A vs. B: Session A scaffolded a full project with tests after code; Session B enforced tests-first TDD in an empty implementation folder.

Hallucination check: Pre-written tests constrained the Green-phase formula; Red prevented shipping untested code.

6 Discussion

1. Red phase failure was intentional and gradable evidence, not an environment bug.
2. Green passed on the first try because tests fully specified behavior.
3. Refactor improved error messages (e.g. “got {CN}”) useful for debugging bad inputs.
4. Next time: add one extra hand-calculated case in tests when trusting a new AI model.

7 Problems encountered

Using a virtual environment from `week3_session_a` for `week3_session_b` worked but is optional; system `python3` and `pytest` were sufficient. Initial confusion that Red “errors” needed fixing was resolved by understanding TDD phase goals.

8 Lessons learned

TDD with AI still requires human verification: run `pytest` locally, keep tests unchanged during Refactor unless physics intentionally changes, and document each phase in `prompt_log.md` with screenshots.

9 Conclusion

Week 3 Session B objectives were met: Red, Green, and Refactor with SCS-CN runoff, seven passing tests throughout Green and Refactor, and documented prompts and outcomes. Submitted artifacts: `week3_session_b/`, `prompt_log.md`, and phase screenshots.

A Submitted project files

A.1 tests/test_runoff.py (Red phase)

tests/test_runoff.py (full file)

```
import pytest
from src.runoff import calculate_runoff

def test_normal_case():
    """P=50 mm, CN=80 → hand-calculated Q 13.81 mm."""
    P = 50.0
    CN = 80
    # S = 25400/80 - 254 = 63.5 mm
    # Ia = 0.2 * 63.5 = 12.7 mm
    # Q = (50 - 12.7)**2 / (50 - 12.7 + 63.5)
    #     = 37.3**2 / 100.8
```

```

# = 1391.29 / 100.8 13.8025
expected = 13.80
result = calculate_runoff(P, CN)
assert result == pytest.approx(expected, abs=0.01)

def test_no_rainfall():
    """P=0 → Q must be 0 regardless of CN."""
    assert calculate_runoff(0.0, 80) == 0.0
    assert calculate_runoff(0.0, 50) == 0.0
    assert calculate_runoff(0.0, 100) == 0.0

def test_rainfall_less_than_initial_abstraction():
    """P < Ia → no runoff (Q = 0)."""
    # CN=80 → Ia=12.7 mm; P=10 < 12.7
    assert calculate_runoff(10.0, 80) == 0.0

def test_impervious_surface():
    """CN=100 → S=0, Ia=0, Q=P."""
    P = 50.0
    result = calculate_runoff(P, 100)
    assert result == pytest.approx(P, abs=1e-9)

def test_negative_rainfall_raises_error():
    """Negative P is physically impossible → ValueError."""
    with pytest.raises(ValueError):
        calculate_runoff(-1.0, 80)
    with pytest.raises(ValueError):
        calculate_runoff(-0.1, 80)

def test_invalid_cn_raises_error():
    """CN outside [1, 100] → ValueError."""
    with pytest.raises(ValueError):
        calculate_runoff(50.0, 0)
    with pytest.raises(ValueError):
        calculate_runoff(50.0, 101)
    with pytest.raises(ValueError):
        calculate_runoff(50.0, -1)

def test_runoff_never_exceeds_rainfall():
    """Physical validation: Q ≤ P for a range of inputs."""
    import itertools
    for P, CN in itertools.product([0, 5, 10, 25, 50, 100, 200],
                                    [1, 30, 50, 70, 80, 90, 100]):
        q = calculate_runoff(float(P), CN)
        assert 0.0 ≤ q ≤ float(P), f"Q={q} out of [0, {P}] for CN={CN}"

```

A.2 src/runoff.py (Green + Refactor)

src/runoff.py (full file)

```

from typing import Union

# SCS-CN constants (mm)
_RETENTION_FACTOR = 25400.0
_RETENTION_OFFSET = 254.0
_INITIAL_ABSTRACTION_RATIO = 0.2

```

```

_MIN_CN = 1
_MAX_CN = 100

def _validate_positive_rainfall(P: Union[int, float]) -> float:
    """Validate and convert rainfall depth P (mm)."""
    if not isinstance(P, (int, float)):
        raise TypeError(f"P must be a number, got {type(P).__name__}")
    P = float(P)
    if P < 0:
        raise ValueError(f"Rainfall depth P cannot be negative, got {P}")
    return P

def _validate_curve_number(CN: Union[int, float]) -> float:
    """Validate and convert curve number CN (dimensionless)."""
    if not isinstance(CN, (int, float)):
        raise TypeError(f"CN must be a number, got {type(CN).__name__}")
    CN = float(CN)
    if not (_MIN_CN <= CN <= _MAX_CN):
        raise ValueError(
            f"Curve number CN must be between {_MIN_CN} and {_MAX_CN}, got {CN}"
        )
    return CN

def _compute_runoff(P: float, CN: float) -> float:
    """SCS-CN runoff depth (pre-validated inputs only)."""
    S = _RETENTION_FACTOR / CN - _RETENTION_OFFSET
    Ia = _INITIAL_ABSTRACTION_RATIO * S
    if P <= Ia:
        return 0.0
    effective_rainfall = P - Ia
    return effective_rainfall ** 2 / (effective_rainfall + S)

def calculate_runoff(P: Union[int, float], CN: Union[int, float]) -> float:
    """Compute direct runoff Q (mm) using the SCS-CN method.

    The Soil Conservation Service Curve Number (SCS-CN) method estimates
    rainfall excess (direct runoff) from total rainfall and a curve number.

    Parameters
    -----
    P : float
        Total rainfall depth (mm). Must be >= 0.
    CN : float
        Curve number in [1, 100]. dimensionless.

    Returns
    -----
    float
        Direct runoff depth Q (mm), clipped so that 0 <= Q <= P.

    Raises
    -----
    ValueError
        If P < 0 or CN is outside [1, 100].

    Formulas
    -----
    S = 25400 / CN - 254          (maximum retention, mm)
    Ia = 0.2 * S                  (initial abstraction, mm)

```



```

If P <= Ia: Q = 0
Else:      Q = (P - Ia)**2 / (P - Ia + S)
"""

P = _validate_positive_rainfall(P)
CN = _validate_curve_number(CN)
Q = _compute_runoff(P, CN)
return max(0.0, min(Q, P))

```

A.3 Prompts

prompt_red.txt

I need to implement the SCS-CN runoff formula for Python 3.8.

Conventions (use in test comments and expected values):

- S = 25400 / CN - 254 (mm)
- Ia = 0.2 * S (mm)
- If P <= Ia: Q = 0
- Else: Q = (P - Ia)² / (P - Ia + S)
- Clip so 0 <= Q <= P
- CN must be in [1, 100]; P >= 0; invalid inputs should raise ValueError

Before writing the function, write comprehensive pytest tests in tests/test_runoff.py
 ↳ that cover:

1. Normal case: P=50 mm, CN=80 (hand-calculate expected Q)
2. Edge case: P=0 (no rainfall)
3. Edge case: P < Ia (no runoff) | e.g. P=10, CN=80
4. Boundary: CN=100 (impervious) with P=50
5. Invalid: negative P (should raise error)
6. Physical validation: Q should never exceed P (loop over several P, CN pairs)

Import from: from src.runoff import calculate_runoff

Write ONLY the test file do NOT create src/runoff.py yet. The function does not exist;
 ↳ pytest must fail with ImportError or test failures (Red phase).

prompt_green.txt

Now implement calculate_runoff(P, CN) in src/runoff.py that will pass all tests in
 ↳ tests/test_runoff.py.

Requirements:

- Python 3.8 compatible type hints (use Union/Optional, not X | Y)
- Follow SCS-CN exactly: S = 25400/CN - 254, Ia = 0.2*S, Q=0 if P<=Ia, else
 ↳ Q=(P-Ia)²/(P-Ia+S), enforce Q <= P
- Handle all edge cases in the tests
- Docstring with formula explanation
- Make all tests pass

[PASTE YOUR FULL tests/test_runoff.py FILE HERE BEFORE SENDING]

prompt_refactor.txt

Review this calculate_runoff function and refactor for readability and better error
 ↳ messages.

Rules:

- Do NOT change numerical outputs (tests are the contract)
- Python 3.8 type hints only
- After each change, all tests in tests/test_runoff.py must still pass

Focus on:

- Code readability
- Clear error messages
- Small helper functions if useful (e.g. validation)
- More Pythonic patterns

[PASTE YOUR CURRENT src/runoff.py HERE BEFORE SENDING]

requirements.txt

pytest>=7.0

B Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```
# Prompt Log - Week 3 Session B

## Red phase

**Prompt used:** `prompt_red.txt`

**Test file:** `tests/test_runoff.py` (7 tests; import `calculate_runoff` from
↳ `src.runoff`)

**pytest result:** `ModuleNotFoundError: No module named 'src.runoff'` Red confirmed (no
↳ implementation yet)

**Notes on failing tests:** Collection error before any test ran; expected for TDD Red.
↳ Tests define normal case P=50 CN=80 (Q13.80), P=0, P<Ia, CN=100, invalid P/CN, and QP
↳ grid.

---

## Green phase

**Prompt used:** `prompt_green.txt` with full `tests/test_runoff.py` pasted

**Implementation:** `src/runoff.py` (single function with inline validation)

**What AI got wrong on first try:** Nothing critical all 7 tests passed on first
↳ implementation.

**How tests caught it:** Tests encode S, Ia, and expected Q; would fail on wrong formula,
↳ missing Q=0 branch, or missing ValueError.

**pytest result:** 7 passed in ~0.01 s

**Sanity check:** P=80, CN=85 43.55 mm; P=50, CN=80 13.80 mm (consistent with Week 1/2
↳ and Week 3 Session A)

---

## Refactor phase

**Prompt used:** `prompt_refactor.txt` with Green-phase `src/runoff.py` pasted

**What changed:**
```

```

- Named constants (`_RETENTION_FACTOR`, `_RETENTION_OFFSET`,
↳ `_INITIAL_ABSTRACTION_RATIO`, CN bounds)
- Helpers: `_validate_positive_rainfall`, `_validate_curve_number`, `_compute_runoff`
- Clearer error messages (include invalid values)
- `effective_rainfall` named intermediate; `calculate_runoff` is a short validate
↳ compute clamp pipeline

**What stayed the same:** SCS-CN math and outputs; all test expectations unchanged

**pytest after refactor:** 7 passed

---

## Hallucination check

**One thing tests prevented or exposed:** Without tests, an AI could use wrong S formula
↳ or skip P1a; the hand-calculated normal case (13.80 mm) and impervious case (Q=P)
↳ lock in correct physics. Red phase forced tests before code so behavior was specified
↳ first.

---

## Lessons learned

TDD with AI: write tests first (Red), implement minimum code (Green), refactor only while
↳ green. Tests are the contract; terminal pytest is mandatory even when the agent says
↳ all pass.

```